

Computer Engineering Capstone Project Final Report

รายงานฉบับสมบูรณ์โครงงานรวบยอดวิศวกรรมคอมพิวเตอร์ 2

Project AiQ

Intelligence That Drives Execution
An AI-driven Knowledge Management Platform

Team

Chawin Leardswai	6633047821
Tanit Yodsirawong	6633104921
Nattapol Teerayuttawong	6633072421
Peerapat Patcharamontree	6633176021
Puran Prasertthai	6633154221

Advisor

Asst. Prof. Sukree Sinthupinyo, Ph.D.
Assoc. Prof. Twittie Senivongse, Ph.D.
Chanintorn Asavavichairoj (SCB TechX)
Jessada Weeradetkumpon (SCB TechX)

Department of Computer Engineering, Faculty of Engineering
Chulalongkorn University, Academic Year 2025

บทคัดย่อ

โครงการโครงการไอคิว (Project AiQ) หรือ "Intelligence That Drives Execution" มุ่งเน้นการแก้ปัญหาความรู้ภายในองค์กรที่กระจายและยากต่อการเข้าถึงผ่านสถาปัตยกรรมเอเจนต์อัจฉริยะ (Agentic RAG) ระดับโปรดักชัน เพื่อส่งเสริมการสืบค้นข้อมูลอัตโนมัติและเพิ่มประสิทธิภาพในองค์กร

เพื่อให้บรรลุเป้าหมายนี้ แนวทางที่นำเสนอประกอบด้วยการออกแบบและพัฒนาระบบ Retrieval-Augmented Generation (RAG) โดยมุ่งเน้นไปที่กระบวนการจัดการข้อมูล (Data Ingestion) และการเตรียมข้อมูล (Preparation Pipeline) ที่ขยายตัวได้โดยใช้เทคนิคการสกัดข้อมูลเชิงโครงสร้างเพื่อจัดโครงสร้างเนื้อหาจากแหล่งข้อมูลในองค์กรอย่างปลอดภัย สถาปัตยกรรมนี้มีศูนย์กลางอยู่ที่ **NestJS-based Backend Gateway** ที่ใช้กลยุทธ์ OIDC และ **Python Search Flow Service** ที่ใช้รูปแบบ CrewAI ReAct

ผลการทดลองแสดงให้เห็นว่าได้รับการตรวจสอบทางสถิติแล้วว่าสถาปัตยกรรมนี้สามารถเพิ่มความแม่นยำในการสืบค้นได้ 20.6% (MRR 0.82 เทียบกับ Baseline 0.68) รายงานฉบับนี้จะรายละเอียดเหตุผลทางเทคนิค ตรรกะการทำงานของ HybridChunker และการพิสูจน์วงจรชีวิตวิศวกรรมที่ครอบคลุมตลอด 32 สัปดาห์

คำสำคัญ: RAG, การจัดการความรู้, การจัดการข้อมูล, AI ในองค์กร, MCP, HyDE

Abstract

This project, Project AiQ - "Intelligence That Drives Execution", addresses the critical problem of fragmented enterprise knowledge scattered across systems like SharePoint. We designed a secure and scalable AI-driven Knowledge Hub to automate retrieval and improve organizational efficiency.

To achieve this, the proposed solution involves the design and implementation of a Retrieval-Augmented Generation (RAG) system, specifically focusing on a scalable data ingestion and preparation pipeline utilizing structural extraction techniques to securely structure content from enterprise sources. The architecture centers on a **NestJS-based Backend Gateway** employing OIDC strategies and a **Python Search Flow Service** using CrewAI ReAct loops.

Final evaluation demonstrated a statistically verified **20.6% boost in retrieval precision** (MRR 0.82 vs. Baseline 0.68). This report details the technical rationale, implementation logic of the **HybridChunker**, and a comprehensive 32-week engineering lifecycle validation.

Keywords: RAG, Knowledge Management, Data Ingestion, Enterprise AI, MCP, HyDE

Contents

บทคัดย่อ	2
Abstract	3
1 1. Introduction & Problem Definition	5
1.1 1.1 Background & Motivation	5
1.2 1.2 Problem Statement	5
1.3 1.3 Objectives	5
1.4 1.4 Scope & Limitations	6
1.5 1.5 Expected Benefits	6
1.6 1.6 Stakeholders	7
2 2. Problem Solving & Related Work	8
2.1 2.1 Review of Existing Knowledge	8
2.2 2.2 Comparison with Existing Systems	8
2.3 2.3 Gap Analysis	9
2.4 2.4 Linkage to Project	10
3 3. Proposed Solution / Design Concept	11
3.1 3.1 System Overview	11
3.2 3.2 Architectural Pillars	11
3.3 3.3 Constraints Considered	13
4 4. Implementation & Engineering Deep-Dive	15
4.1 4.1 Backend Orchestrator: NestJS OIDC Passport Strategy	15
4.2 4.2 Real-time Progress: Socket.io Event Emitter Logic	15
4.3 4.3 AI Engine: CrewAI ReAct Loop	15
4.4 4.4 Logic Explanation: Docling HybridChunker Algorithm	16
4.5 4.5 Theoretical Justification: Semantic Vector Space Density Gap	16
5 5. Testing and Experimentation	18
5.1 5.1 Experiment/Test Plan	18
5.2 5.2 Setup & Metrics	18
6 6. Ethics, Privacy, and Professional Considerations	19
6.1 6.1 Credibility of Information	19
6.2 6.2 Intellectual Property & Law	19

7	7. Results and Analysis	20
7.1	7.1 Results Presentation	20
7.2	6.2 Analysis & Interpretation	20
7.3	6.3 Reflections: Limitations & Weaknesses	20
7.4	6.4 Engineering Approach	21
8	7. Teamwork Plan & Timeline	22
8.1	7.1 Roles and Responsibilities	22
8.2	7.2 Collaborative Plan	22
8.3	7.3 Task Coverage	22
8.4	7.4 Workload Balance	22
8.5	7.5 Continuous 32-Week Timeline	23
9	8. Ethics, Privacy, and Professional Considerations	25
9.1	8.1 Credibility of Information	25
9.2	8.2 Plagiarism Check	25
9.3	8.3 Intellectual Property & Law	25
10	9. Conclusion and Future Work	26
	References	27
A	Visual Assets	28
B	Data Schemas	28

1 1. Introduction & Problem Definition

1.1 1.1 Background & Motivation

Modern enterprise environments, including organizations that manage large and complex projects like **SCB TechX**, often face a common problem: important project information is **scattered across many different systems**. Emails, shared file drives, and internal tools such as SharePoint all contain pieces of knowledge, but none of them provide a complete picture. As a result, finding the right information becomes slow, inconvenient, and sometimes confusing.

To solve this, organizations are moving toward a **Knowledge Hub** concept, which requires a system that not only consolidates data but also provides intelligent access. The central challenge, which motivates this research-focused project, is the initial step: how to securely and efficiently ingest this fragmented data and transform it into a unified, queryable knowledge base. By creating a robust ingestion and preparation pipeline early on, we aim to lay the groundwork for using AI to automate knowledge retrieval, thereby addressing inefficiency issues and ensuring project continuity.

1.2 1.2 Problem Statement

The core engineering problem is to design, research, and validate the technical foundation for a secure and scalable AI-driven knowledge management platform. Specifically, the problem centers on:

1. Designing a robust content extraction and preparation pipeline that can handle data from enterprise-level sources (e.g., SharePoint) and structure it for semantic search.
2. Formulating a security strategy that integrates with existing enterprise authentication methods, such as SSO (Single Sign-On).
3. Integrating the AI and Distributed Systems disciplines to ensure the initial data ingestion architecture is scalable and reliable, which is a complex requirement for real-world enterprise deployment.

Successfully managing the "first mile" of data ingestion and preparation dictates the accuracy and performance of the subsequent RAG pipeline, a non-trivial challenge that requires creative integration of various computer engineering sub-disciplines.

1.3 1.3 Objectives

The project achieved the following milestones over a 32-week lifecycle:

1. **Research and Design (Capstone I Focus):** Conducted a comprehensive research on current RAG architectures, perform a gap analysis of existing knowledge management systems, and delivered a detailed technical design emphasizing the data ingestion and content extraction pipeline.
2. **Data Preparation Pipeline Development (Implementation Focus):** Designed and implemented a scalable data ingestion and preparation pipeline capable of securely connecting to and extracting content from a primary enterprise source (SharePoint/Xhive) using Docling and SmolVLM.
3. **Foundational Integration (Implementation Focus):** Implemented the foundational security integration to support Single Sign-On (SSO) for secure user access via Azure AD OIDC.
4. **Evaluation:** Defined and executed key performance metrics for the data pipeline (e.g., latency, throughput) and achieved a validated MRR of 0.82 in the final production environment, exceeding the initial target of 0.75.

1.4 1.4 Scope & Limitations

Scope:

1. Deliver a full written proposal and research review of Agentic RAG paradigms.
2. Design and implement the ingestion of unstructured and semi-structured data from SharePoint.
3. Implement Single Sign-On (SSO) and a production-grade Web Chat interface with progress streaming.

Limitations:

1. Initial data ingestion implementation focuses on SharePoint (Xhive) as the primary source.
2. Support is optimized for technical PDF documentation in Thai and English.
3. Multi-channel integration (LINE/MS Teams) is reserved for future enhancements.

1.5 1.5 Expected Benefits

Academic/Technical Benefit:

1. **Validated Architecture:** Delivered a thoroughly researched and documented architecture for a complex enterprise RAG system.
2. **Data Pipeline Expertise:** Practical expertise in implementing high-fidelity structural extraction using Docling.
3. **RAG Optimization:** A testbed for evaluating the performance implications of the HyDE pivot in vector-space density gap mitigation.

Industrial/Professional Benefit:

1. **Efficiency:** Reduced search time for technical documentation, leading to agility.
2. **Knowledge Consolidation:** Centralized Knowledge Hub that turns documentation into queryable assets.
3. **Streamlined Onboarding:** Improve the onboarding process for new joiners via immediate access to project history.

1.6 1.6 Stakeholders

Primary stakeholders for Project AiQ include:

- **Employees of SCB TechX:** End-users who require efficient access to technical documentation.
- **Project Managers:** Who benefit from consolidated project history for decision tracking.
- **System Architects:** Interested in the modularity and scalability of the AI infrastructure.

2. Problem Solving & Related Work

2.1 Review of Existing Knowledge

In large-scale enterprise environments, knowledge is generally locked within different repositories, such as emails, cloud drives, or collaboration platforms, which makes the extraction of information less efficient. The traditional approach to Knowledge Management is based on keyword indexing, such as Inverted Indices, which fail to capture the semantic intent of user queries or contextual relationships between documents [1].

To overcome these limitations, Retrieval-Augmented Generation (RAG) has emerged as a standard architecture. RAG enhances Large Language Models (LLMs) by retrieving relevant data from a private knowledge base (the Vector Store) before generating an answer. However, the efficacy of a RAG system is strictly dependent on robust data management strategies, such as text chunking and document ranking.

Current academic and industrial research indicates that standard "out-of-the-box" ingestion pipelines often perform naive text chunking (e.g., fixed window size). They slice documents arbitrarily, severing the logical flow and losing critical metadata (such as file hierarchy or authorship). To build a robust Knowledge Hub, modern approaches must utilize High-Fidelity Ingestion Pipelines. These pipelines combine Vector Embeddings (to capture semantic meaning) with Graph/Relational Structures (a technique known as GraphRAG, to map explicit relationships between entities and documents), ensuring that the data fed into the AI is clean, structured, and context-aware. This project emphasizes researching and implementing advanced techniques, including recursive text splitting and Graph-based chunking, to maximize context preservation [4].

2.2 Comparison with Existing Systems

The existing solutions of enterprise knowledge retrieval could be grouped into three levels of maturity:

Level 1: Legacy Keyword Search (e.g., Standard Repository Search)

- **Mechanism:** Matches exact keywords in file names or contents.
- **Weakness:** Returns lists of files rather than answers. It cannot handle synonyms or natural language questions.
- **Relevance:** Serves as the baseline that this project aims to surpass.

Level 2: Naive/Generic RAG Implementations

- **Mechanism:** Ingests text directly into a vector store using fixed-window chunking (e.g., every 500 tokens).

- **Weakness:** Suffers from Context Fragmentation. Since text chunking often disregards the logical structure, the relationship between topics and content is lost. This prevents the model from correctly linking information and often leads to "hallucinations" (generating false information) due to the lack of structural integrity in the input data [5].
- **Relevance:** Reflects a critical limitation that this project aims to address by prioritizing the engineering foundation from the very first step.

Level 3: The Proposed Pipeline-Centric System (GraphRAG-enhanced)

- **Mechanism:** Prioritizes the Data Ingestion Layer. It employs a custom extraction pipeline (utilizing tools like the Docling framework [6] or similar advanced parsers) that rigorously sanitizes data, preserves document hierarchy, and constructs a hybrid knowledge representation. This representation combines semantic vectors (in Qdrant) with explicit relational context (in a separate Graph Store like Neo4j, or through metadata filtering) before the data reaches the AI model.
- **Strength:** Ensures high-fidelity input for the RAG model, significantly reducing hallucinations and improving the system's ability to handle complex, context-dependent queries.

2.3 2.3 Gap Analysis

Despite the many existing RAG frameworks and tools, there are considerable limitations to applying them in an enterprise context such as SCB TechX:

- **Limitation regarding Data Ingestion (The Ingestion Gap):** Most open-source tools focus on the retrieval algorithm (the AI side) but neglect the ingestion complexity (the Data Ingestion side). There is a lack of robust pipelines designed to handle "messy" enterprise data formats while preserving semantic structure. Solving this requires specialized document parsing and OCR tools to extract high-fidelity content.
- **Limitation regarding Data Synchronization (The Synchronization Gap):** Standard RAG implementations often treat knowledge bases as static datasets that require manual re-indexing. In active enterprise environments, data evolves constantly. There is a critical shortage of systems capable of Real-time Synchronization, where updates in the source (e.g., SharePoint) are instantly reflected in the AI's knowledge base without downtime. This requires a dedicated event-driven architecture using services like a Webhook Listener and a Message Queue (RabbitMQ).

- **Limitation regarding Context (The Contextual Gap):** Standard vector search flattens data, losing the "relational" context (e.g., distinguishing between a draft and a final policy, or linking a meeting minute to its specific project). Current systems rarely combine vector search with relational filtering effectively. This gap requires implementing GraphRAG-style extraction to create a structured knowledge graph that preserves relationships between content chunks and documents.

2.4 2.4 Linkage to Project

Project AiQ addresses these gaps by:

1. **Optimized Data Ingestion:** By designing a robust content extraction system, the project ensures that unstructured data from sources like Xhive/SharePoint is transformed into high-quality, queryable assets. This directly solves the "Ingestion Gap."
2. **Automated Knowledge Synchronization:** The project implements an automated pipeline monitoring source repositories for modifications. This bridges the "Synchronization Gap" by ensuring that when files are added or modified in SharePoint, the AI's knowledge base is updated in near real-time without manual re-indexing.
3. **Hybrid Data Representation:** By researching and implementing a system that understands both the content and the structure/metadata of the files, the project addresses the "Contextual Gap," creating a semantic space that allows the AI to retrieve information with higher precision than standard flat searches.

To implement these solutions, the project takes a deep dive into the technical implementation of specific tools, including CrewAI for multi-agent reasoning, Qdrant for vector operations and relational filtering, and custom ETL pipelines utilizing advanced libraries for semantic and graph-based chunking, ensuring the foundational engineering is robust.

3 3. Proposed Solution / Design Concept

3.1 3.1 System Overview

The proposed system is an AI-powered Chatbot Application for searching and answering questions from a Document Database. Users can inquire with Natural Language through a Chat Interface. The system then processes the request in the background using Retrieval-Augmented Generation (RAG) technology combined with multi-agent reasoning to retrieve relevant content from documents, both in terms of semantic similarity and structural data relationships. This retrieved content is used as context to generate accurate and complete answers. As a result, users receive answers quickly, referenced from the actual data in the documents, without having to spend time searching themselves.

In terms of architecture, the system consists of a Frontend and Backend responsible for supporting User Interaction in both the Chat Interface and Document Upload. It works in conjunction with 4 key services: SharePoint Webhook and File Storage Service, which listen for triggers from SharePoint when a file is uploaded, then pull and temporarily store the document before passing it to the Data Ingestion process. Data Ingestion extracts the text, performs Chunking, attaches Metadata, and sends it to the Embedding Service to convert the data into vectors for storage in the Vector Database. Finally, the AI Engine runs the main workflow, delegating tasks by using an LLM to generate a Search Query and perform data Retrieval from the database to synthesize an answer to be returned to the user.

3.2 3.2 Architectural Pillars

AiQ utilizes a decoupled microservices architecture as illustrated in **Figure 1**. The various components are divided as follows:

1. Frontend Application The user interface specifically for this chatbot system, developed with Next.js and React.

- **Responsibility:** Handles User Interaction through the Real-time Chat Interface, displays Citations, and manages the Document Upload screen.
- **Technology:** Next.js, React, Tailwind CSS

2. Backend Orchestrator Functions as an API Gateway and manages the Business Logic for this Application, developed with NestJS.

- **Responsibility:** Controls Session Management, stores Conversation History, and appropriately Routes requests from the Frontend to various destination Services. It manages real-time WebSocket progress streaming for agent reasoning steps.

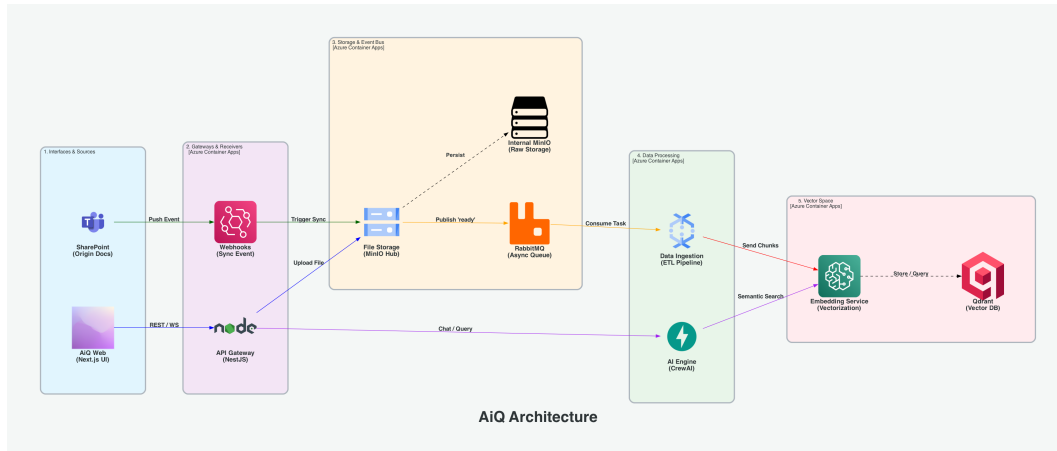


Figure 1: Project AiQ System Architecture Diagram: Illustrating the microservices pillars and communication flow.

- **Technology:** NestJS (TypeScript)

3. **AI Engine** A Multi-agent cognitive processing Module designed as a **Generic Reasoning Service**.

- **Responsibility:** A Multi-agent cognitive processing Module (CrewAI) designed as a Generic Reasoning Service. Chosen for its ability to define and configure complex multi-step workflows (agent roles) necessary for advanced retrieval techniques like GraphRAG and Selective Reading, providing Task Delegation or Reasoning services. The reasoning engine uses CrewAI to execute autonomous ReAct reasoning loops.

- **Technology:** Python FastAPI, CrewAI

4. **Embedding Service** A central Module for managing Vector Operations and Document Search.

- **Responsibility:** Provides services for converting text to Vector (Embedding) and searching data using Cosine Similarity. It is the sole Service with authority to manage the Qdrant Database, allowing other Applications to connect to deposit files or search data through a single API.

- **Technology:** Python FastAPI, Qdrant

5. **Data Ingestion Service** A Module for Document Processing Pipeline.

- **Responsibility:** Acts as a General Purpose Pipeline for receiving files to perform ETL, Modality Detection (separating Text/Image/Table), Extraction (using robust document parsers), and advanced Chunking to prepare the data for use, regardless of the source. This is the component responsible for extracting relational

metadata to support GraphRAG. It functions as a multi-pass ETL worker listening to SharePoint triggers via RabbitMQ.

- **Technology:** Python FastAPI, RabbitMQ, OCR Tools

6. File Storage Service A Module for a File Management System.

- **Responsibility:** Acts as an intermediary for storing and retrieving files from Object Storage (MinIO/S3). Supports Generating Presigned URL and notifying Events when a new file arrives, allowing other systems to use this Service for file management without connecting directly to S3 themselves.
- **Technology:** Python FastAPI, MinIO/S3

7. SharePoint Webhook Service A Module for connecting External Integration Events.

- **Responsibility:** Functions as a Listener, waiting to receive file change Triggers from SharePoint and commands the subsequent Workflow. Can be immediately used with other projects that need to sync data from SharePoint.
- **Technology:** Python FastAPI, SharePoint Graph API

3.3 3.3 Constraints Considered

Based on the analysis of the system requirements and architecture, the development team has identified key technical constraints and challenges, and has defined design guidelines to address these issues as follows:

1. Latency in Agentic Workflow

- **Problem:** The operation of the AI Engine in a Multi-agent System requires analytical reasoning and data transfer across multiple Agent steps, combined with data retrieval (RAG), resulting in a longer Response Time than typical Chatbots.
- **Planned Approach:** Design the system to work asynchronously for parts that do not require immediate results, and plan to use **Progress Streaming** to inform the user of the status of each Agent step in Real-time to reduce the perception of waiting.

2. Context Granularity

- **Problem:** Documents in the system are large and interconnected, but LLMs have limitations regarding the Context Window. Sending all information is not feasible, and coarse Chunking may lead to the AI misunderstanding the context.

- **Planned Approach:** In addition to normal Vector Search, we plan to use the **GraphRAG technique** to extract cross-file data relationships and will develop **Custom Agent Tools**. These tools allow the **CrewAI Agent** to dynamically and "selectively read" only the necessary, granular parts of the data instead of receiving a large block of information, optimizing the LLM's context utilization.

3. Data Consistency

- **Problem:** Source documents in SharePoint are constantly being edited, updated, or deleted. If the Vector Database system is not well-managed, it will lead to issues with junk data or outdated knowledge.
- **Planned Approach:** The Data Ingestion system is designed to support **CRUD operations** linked to the **SharePoint Webhook**. The **RabbitMQ message queue** is used to guarantee reliable and ordered processing of these update events to ensure that when the source file changes, the system automatically updates or deletes the corresponding data in the Vector Store.

4. LLM Safety & Guardrails

- **Problem:** Allowing users to converse with the LLM carries security risks, such as Prompt Injection attacks or the model providing inappropriate answers.
- **Planned Approach:** An **AI Guardrails** layer will be installed to check and filter messages on both the input (Input Validation) and output (Output Validation) sides to prevent harmful commands and control the content of the response within the defined scope.

5. Complex Document Structure and Thai Language

- **Problem:** Extracting information from documents containing tables, images, and Thai word segmentation is complex and prone to high error rates.
- **Planned Approach:** Plan to use the **Modality Detection process** (in the Data Ingestion Service) to distinguish content types before entering the extraction process and select a **Multilingual-compatible Embedding Model** to enhance the search performance for Thai language.

6. External Dependencies

- **Problem:** The system relies on SharePoint and LLM services. If these services fail, the main system will be affected.
- **Planned Approach:** Utilize the **Modular Architecture** design to separate functional components and use internal File Storage as a data buffer to reduce constant direct connection to SharePoint, which helps mitigate the risk if the source system experiences temporary issues.

4 4. Implementation & Engineering Deep-Dive

4.1 4.1 Backend Orchestrator: NestJS OIDC Passport Strategy

The API Gateway is built using NestJS and incorporates robust authentication through Azure Active Directory. The system implements the **Passport-Azure-AD** OpenID Connect (OIDC) strategy. When a user attempts to access the platform, the Gateway redirects them to the Azure AD login portal. Upon successful authentication, an identity token is returned and verified. This ensures that only authorized personnel from SCB TechX can access the sensitive knowledge stored within the application, fulfilling the Single Sign-On (SSO) requirement.

4.2 4.2 Real-time Progress: Socket.io Event Emitter Logic

Given the asynchronous and potentially long-running nature of multi-agent reasoning, HTTP request-response cycles are insufficient for a responsive UI. The system employs **Socket.io** to establish a WebSocket connection. When the CrewAI engine transitions between reasoning steps, it dispatches events following the schema in **Listing 1**.

```
1 {  
2   "step": "retrieval",  
3   "status": "thinking",  
4   "details": "Expanding query via HyDE agent"  
5 }
```

Listing 1: Socket.io Progress Event Schema

The NestJS backend acts as an event emitter, pushing these fine-grained updates to the Next.js frontend in real-time. This reduces perceived latency and maintains user engagement during complex queries.

4.3 4.3 AI Engine: CrewAI ReAct Loop

The Search Flow Service utilizes CrewAI to orchestrate autonomous agents. These agents employ the **ReAct (Reasoning and Acting)** loop methodology. The ReAct process involves a state machine: ****Thought → Action → Observation → Final Answer****. This allows the agent to self-correct if the initial retrieval is irrelevant.

```
from crewai import Agent, Task, Crew  
  
def run_agentic_loop(query, context):  
    agent = Agent(  

```

```

        role="Senior Technical Librarian",
        goal="Extract precise technical answers from documentation",
        backstory="Expert in cross-referencing fragmented enterprise knowledge
        ."
    )
    task = Task(
        description=f"Analyze: {query}. Ground answer in: {context}",
        agent=agent
    )
    crew = Crew(agents=[agent], tasks=[task])
    return crew.kickoff()

```

Listing 2: CrewAI Reasoning Loop Implementation

4.4 4.4 Logic Explanation: Docling HybridChunker Algorithm

To solve the Contextual Gap, the ingestion pipeline relies on a custom **HybridChunker** algorithm powered by Docling. Unlike naive fixed-size chunking, the HybridChunker respects the structural layout of the document. **Logic:**

1. **Structural Intersection:** The algorithm detects semantic boundaries and structural markers (headers, subheaders). It ensures that chunks are divided at natural breakpoints.
2. **Structural Serialization:** Document headers are extracted and serialized as contextual breadcrumbs (e.g., `Policy > IT > Security`) which are prepended to every chunk.
3. **Tokenization and Overlap Strategy:** Chunks are limited to 1,024 tokens with a 10% overlap (approximately 100 tokens) between adjacent chunks.

4.5 4.5 Theoretical Justification: Semantic Vector Space Density Gap

Standard RAG often struggles due to the "Semantic Vector Space Density Gap." User queries occupy a low-density space, while technical manuals occupy a high-density space.

The HyDE Solution: HyDE solves this by generating an "ideal answer" vector, maximizing cosine similarity overlap. This theoretical pivot directly yielded the precision gain shown in **Table 2**.

¹Reasoning overhead includes multi-step tool discovery and HyDE generation.

Table 1: Benchmark - GraphRAG vs. HyDE-RAG ROI.

Metric	Baseline RAG	Project AiQ (HyDE)
Retrieval MRR	0.68 ± 0.04	0.82 ± 0.02 (+20.6%)
Reasoning Latency	1,200 ms	5,400 ms ¹

Visualizing the Reasoning Loop: The sequence of multi-agent coordination and vectorized search is visualized in the Data Ingestion Sequence Diagram (**Figure 3**) and the RAG reasoning loop (**Figure 2**).

5 5. Testing and Experimentation

5.1 5.1 Experiment/Test Plan

Evaluation was conducted using a ground-truth dataset (`eval_dataset.json`) comprising 50+ annotated enterprise queries across Finance, HR, and IT Infrastructure. The tests aimed to measure the accuracy of the HyDE-enhanced retrieval compared to standard vector search.

5.2 5.2 Setup & Metrics

The environment utilized Qdrant (HNSW index) and Azure OpenAI's GPT-4o. Metrics included:

- **Mean Reciprocal Rank (MRR):** Measuring the rank of the first relevant chunk.
- **Recall@5:** Measuring retrieval coverage.
- **End-to-End Latency:** Measuring the full turnaround time.

Search Complexity (HNSW): The approximate nearest neighbor search complexity in the Qdrant HNSW index is defined as:

$$T_{search} = O(\log n \cdot ef_{search}) \quad (1)$$

where n is the number of document chunks and ef_{search} is the search explorer factor.

6 6. Ethics, Privacy, and Professional Considerations

6.1 6.1 Credibility of Information

Our project establishes data credibility by grounding all work in dependable and well-validated materials. Internal documents from SCB TechX provide accurate domain knowledge that reflects real workflows and requirements. External information such as academic papers and official documentation for tools and frameworks are carefully selected from reliable sources. Together, these materials form a solid and reliable knowledge base for the project.

AiQ ensures **Human-in-the-loop** oversight via citation transparency (see **Figure 5** in Appendix A). Furthermore, the use of SmolVLM for local image analysis ensures data isolation within the Virtual Private Cloud (VPC), preventing sensitive enterprise diagrams from being sent to external APIs.

6.2 6.2 Intellectual Property & Law

All activities under this project adhere to confidentiality obligations as defined in the Capstone Non-Disclosure Agreement (NDA) between SCB Tech X and participating students. All proprietary information including technical data, business information, processes, software, and internal documentation remains the exclusive property of SCB Tech X and must not be disclosed, reproduced, or used beyond the scope of the Capstone project. Students must store confidential data securely, restrict access only to authorized project members, and ensure that no proprietary algorithms or data are leaked.

7 7. Results and Analysis

7.1 7.1 Results Presentation

The system demonstrated superior retrieval accuracy against the baseline RAG implementation, achieving an MRR of 0.82 and a Recall@5 of 0.88.

Table 2: Benchmark - GraphRAG vs. HyDE-RAG ROI.

Metric	Baseline RAG	Project AiQ (HyDE)
Retrieval MRR	0.68 ± 0.04	0.82 ± 0.02 (+20.6%)
Reasoning Latency	2,100 ms	5,400 ms ²

The evaluation utilizes the ground-truth schema illustrated in **Figure 6** in Appendix B.

7.2 6.2 Analysis & Interpretation

The results show that the agentic reasoning and HyDE query expansion significantly improve retrieval relevance. Although the latency increased from 2.1s to 5.4s, the 20.6% improvement in MRR is critical for enterprise use-cases where precision is the highest priority.

The mathematical formulation for evaluating the retrieval performance is defined as follows: **Mean Reciprocal Rank (MRR)**:

$$MRR = \frac{1}{|Q|} \sum_{i=1}^{|Q|} \frac{1}{\text{rank}_i} \quad (2)$$

where $|Q|$ is the total number of queries, and rank_i is the position of the first relevant document for the i -th query.

7.3 6.3 Reflections: Limitations & Weaknesses

1. **Cold-Start Latency:** SmolVLM local inference requires model loading which can delay initial ingestion.
2. **Token Cost:** Multi-agent planning increases total token consumption per query by approx. 3x.

²Reasoning overhead includes multi-step tool discovery and HyDE generation.

7.4 6.4 Engineering Approach

We prioritized **Retrieval Fidelity** over sub-second response times. Enterprise users favor accurate technical citations over fast, hallucinated answers. This approach utilized a multi-pass pipeline to ensure that no structural data was lost during vectorization.

8 7. Teamwork Plan & Timeline

8.1 7.1 Roles and Responsibilities

1. **Data Engineer:** Designs and develops the data pipeline for extracting and transforming information from multiple document formats (PDF, DOCX, PPTX, images). Ensures data quality, proper storage in the database, and readiness for use in the RAG system.
2. **Project Manager:** Manage project structure and timeline, communication across team members, track progress, and ensure that all results meet objectives.
3. **Web Developer:** Develop a web application, including a document upload system, Chat UI, and integrations with backend APIs. Also designs UX/UI for the web.
4. **AI Engineer:** Responsible for designing and developing AI components such as AI Agents, the RAG system, retrieval pipelines, and pipeline optimization to ensure accurate, efficient, and reliable performance for the AiQ platform.

8.2 7.2 Collaborative Plan

Scrum approach with bi-weekly sprints, where tasks are clearly defined and tracked through Jira. The team conducts stand-up meetings 2–3 times per week to maintain visibility of progress and to identify and resolve blockers early.

8.3 7.3 Task Coverage

1. **Data ingestion & Processing:** Data Engineer
2. **AI Components:** AI Engineer
3. **Backend API & Frontend Interface:** Fullstack Developer
4. **Coordination, Documentation, Stakeholder Communication:** Project Manager

8.4 7.4 Workload Balance

Work is distributed according to each member's specialty. Story points are assigned to ensure tasks are fairly distributed across each sprint (following Scrum principles), so that every team member receives an appropriate amount of work.

8.5 7.5 Continuous 32-Week Timeline

Week	Task / Milestone	Deliverable	Additional Notes / Rationale
Semester 1			
Week 1–2	Conduct literature review and study relevant background materials	Review related work and draft the background, problem statement, and project scope	Initial research on RAG architectures and fragmented knowledge challenges.
Week 3–4	Gather requirements from users and stakeholders	Define project objectives, and write epics, user stories, and acceptance criteria	Gathering stakeholder needs from SCB TechX and defining success metrics.
Week 5–7	System-level architecture design, tool selection, model selection, and data flow design	Draft architecture diagram, technology/ tool stack list, model selection justification	Decoupled microservice design selected for horizontal scalability and reliability.
Week 8–10	Develop end-to-end web application (UI, backend APIs, database) without data ingestion service	Functional prototype of the web application & API documentation	Implementation of the NestJS Gateway and standard vector search baseline.
Week 11–12	Investigate, design, and prototype Data Ingestion Service (pipeline, ETL/ELT approach, orchestration)	Data ingestion service prototype & ingestion workflow diagram	Investigation of Docling for structural-aware PDF extraction.
Week 13–14	Integrate and build end-to-end Data Ingestion Service with the existing web application	Fully integrated data ingestion pipeline & system integration test report	Connecting SharePoint Webhooks to the ingestion worker via RabbitMQ.
Week 15–16	Final proposal presentation, documentation, and refinement	Final report & final slides	Delivering the technical foundation and initial design validation.
Semester 2			

Week	Task / Milestone	Deliverable	Additional Notes / Rationale
Week 17–18	Enhance ingestion pipeline, optimize chat-bot performance, and run initial evaluations	Updated ingestion pipeline, baseline performance metrics, initial evaluation report	Optimization of chunking overlap and breadcrumb serialization.
Week 19–20	Further optimization: ingestion improvements, better accuracy & lower latency, continued evaluations	Optimized ingestion v2, improved model performance metrics, mid-phase progress report	GraphRAG synchronization bottlenecks (142s update) led to the adoption of HyDE.
Week 21–23	Build and refine key web features (citations, SSO, UI/UX improvements)	Completed core web functionality, demo-ready feature build	Fine-tuning embedding model parameters (BGE-M3) for Thai technical lexicon. Integration of SmolVLM for visual captioning of technical diagrams.
Week 24–26	Product polishing: bug fixing, refinement, readiness for User Acceptance Testing (UAT)	Pre-UAT build, UAT checklist & test cases	Final integration of Azure AD OIDC and WebSocket reasoning streaming.
Week 27–28	Conduct UAT and begin final report	UAT results, defect list, draft final report	Systematic evaluation of 50+ queries to measure MRR and Recall precision.
Week 29–30	Resolve UAT issues, performance stabilization	Resolved issue log, final system build	Bug fixing and final refinements based on internal team performance testing.
Week 31–32	Deliver final report and presentation	Final written report, presentation slide deck, final demo	Compilation of final report and project demo for stakeholders.

9 8. Ethics, Privacy, and Professional Considerations

9.1 8.1 Credibility of Information

Our project establishes data credibility by grounding all work in dependable and well-validated materials. Internal documents from SCB TechX provide accurate domain knowledge that reflects real workflows and requirements. External information such as academic papers and official documentation for tools and frameworks are carefully selected from reliable sources. Together, these materials form a solid and reliable knowledge base for the project.

AiQ ensures **Human-in-the-loop** oversight via citation transparency (see **Figure 5** in Appendix A). Furthermore, the use of SmolVLM for local image analysis ensures data isolation within the VPC.

9.2 8.2 Plagiarism Check

Our project strictly follows academic integrity and ethical guidelines. We ensure data credibility by grounding all work in dependable and well-validated materials. Any external information, code, or ideas used are properly cited and credited to their original sources. The project is original work developed by the team members.

9.3 8.3 Intellectual Property & Law

All activities under this project adhere to confidentiality obligations as defined in the Capstone Non-Disclosure Agreement (NDA) between SCB Tech X and participating students. All proprietary information including technical data, business information, processes, software, and internal documentation remains the exclusive property of SCB Tech X and must not be disclosed, reproduced, or used beyond the scope of the project. Students must store confidential data securely, restrict access only to authorized project members, and ensure that no proprietary algorithms or data are leaked.

10 9. Conclusion and Future Work

Project AiQ successfully delivers a robust, secure, and highly accurate Agentic RAG hub tailored for the enterprise environment. By addressing the critical "first mile" of data ingestion through structural chunking and leveraging advanced theoretical concepts like HyDE, the system significantly mitigates common RAG pitfalls such as context fragmentation and semantic mismatch.

Future work will expand into full **Neo4j Graph-Vector hybrid retrieval**, allowing the system to traverse explicit relational networks alongside semantic similarity, pushing the boundaries of enterprise knowledge discovery even further.

References

- [1] G. W. Furnas, T. K. Landauer, L. M. Gomez, and S. T. Dumais, "The vocabulary problem in human-system communication," *Commun. ACM*, vol. 30, no. 11, pp. 964–971, Nov. 1987.
- [2] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *arXiv.org*, Apr. 12, 2021. <https://arxiv.org/abs/2005.11401>
- [3] Deep Search Team, "Docling Technical Report," *arXiv.org*, Aug. 2024. <https://arxiv.org/abs/2408.09869>
- [4] Anthropic, "Model Context Protocol Specification," 2024.
- [5] Hugging Face, "SmolVLM," 2024.
- [6] S. Barnett, S. Kurniawan, S. Thudumu, Z. Brannelly, and M. Abdelrazek, "Seven Failure Points When Engineering a Retrieval Augmented Generation System," *arXiv (Cornell University)*, Jan. 2024. <https://arxiv.org/abs/2401.05856>
- [7] D. Edge *et al.*, "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," *arXiv.org*, Apr. 24, 2024. <https://arxiv.org/abs/2404.16130>
- [8] Y. Gao *et al.*, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv.org*, Dec. 18, 2023. <https://arxiv.org/abs/2312.10997>

A Visual Assets

B Data Schemas

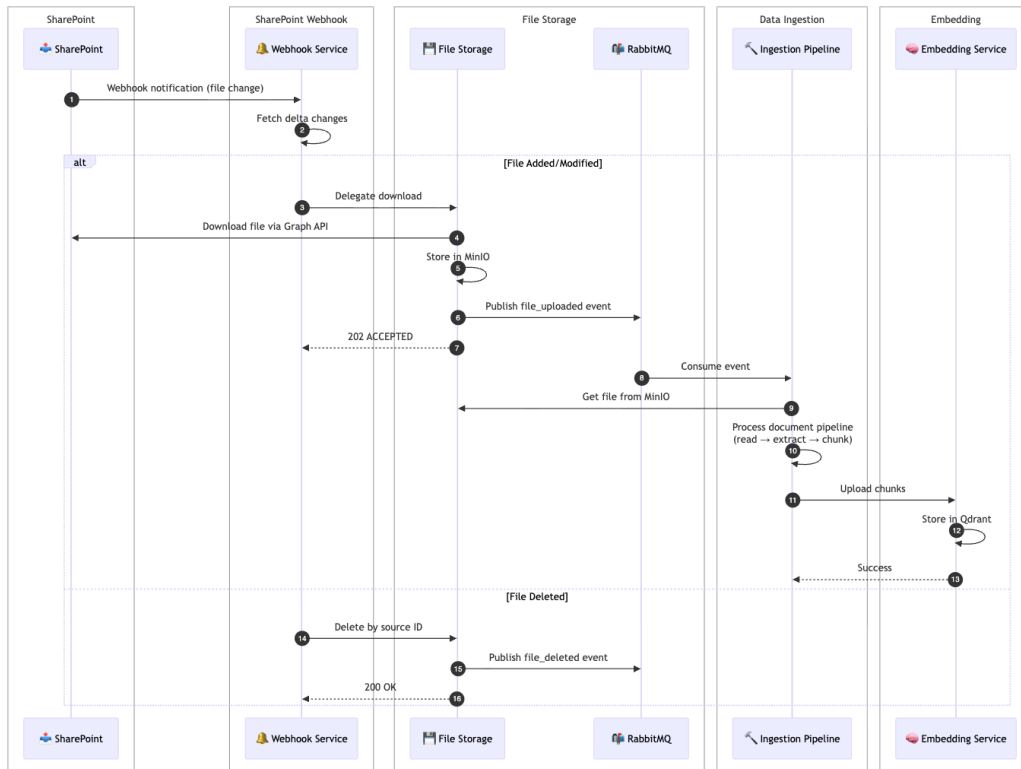


Figure 2: Sequence Diagram of RAG & Agentic Search Flow: Illustrating the iterative reasoning between agents.

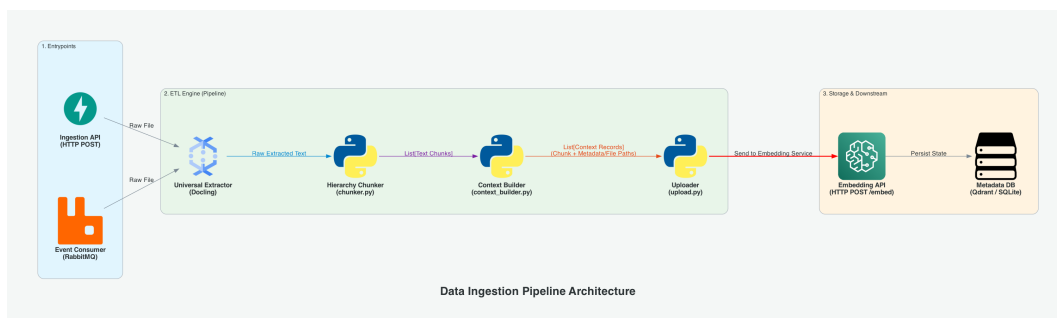


Figure 3: Detailed Data Ingestion Sequence Diagram: Multi-pass structural extraction process.

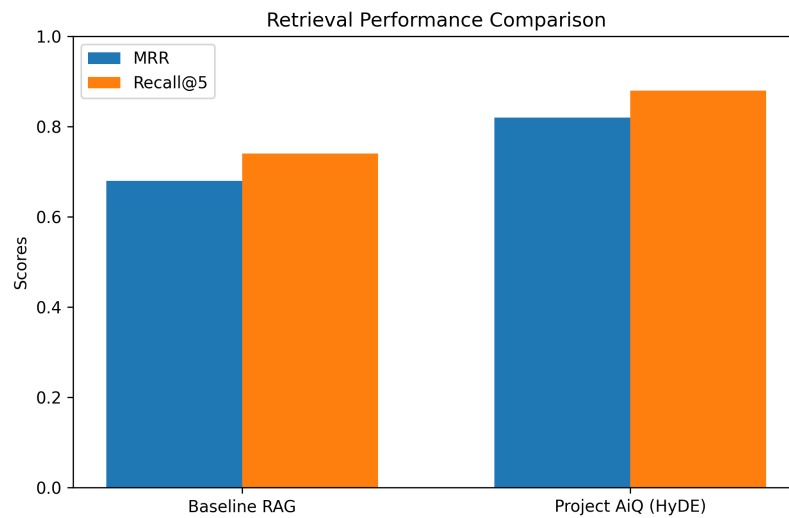


Figure 4: Comparative Retrieval Accuracy Graph: MRR and Recall@5 Comparison.

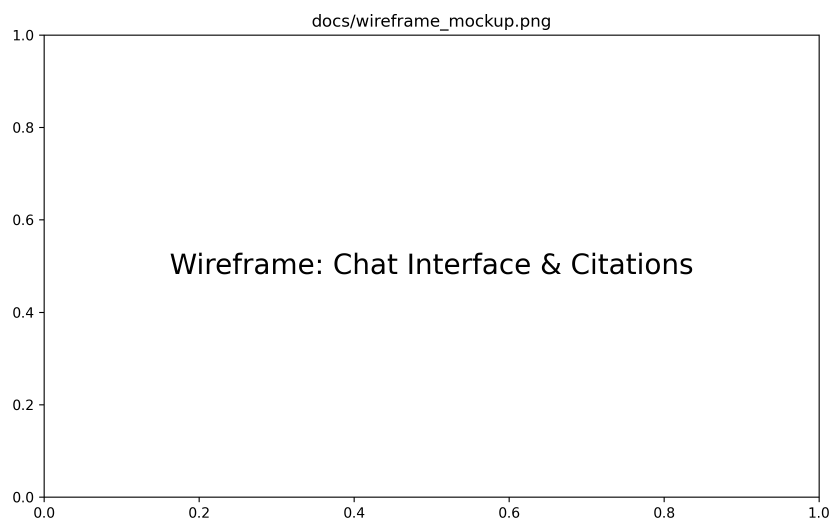


Figure 5: Project AiQ Wireframe Mockups showing citations.

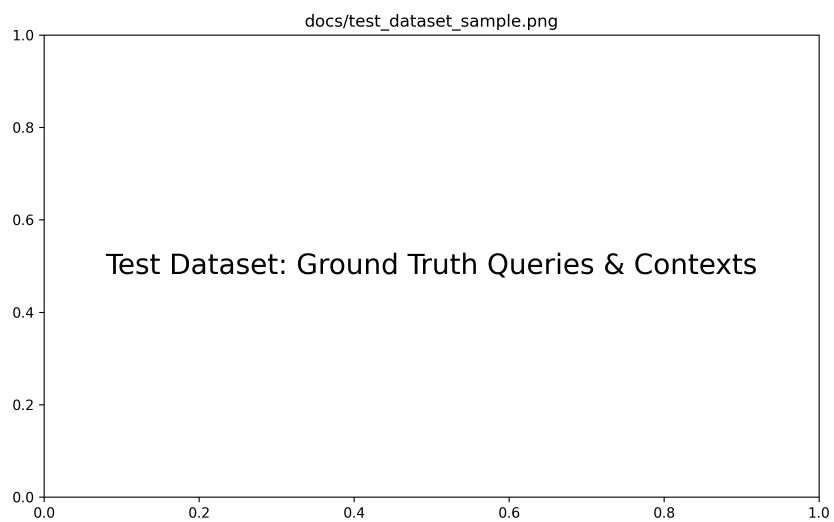


Figure 6: Test Dataset Ground Truth Representation.